

SBT-DF203: Basic Networking Skills for Digital Forensics

LAB 7: DNS Introduction and Traffic Analysis

**INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY
(ICDFA)**

**SBT-DF203 · PRACTICAL LABORATORIES · LAB 7
DNS INTRODUCTION AND TRAFFIC ANALYSIS**

Field	Detail
Student	Ibrahim Disih Garba
Registration No.	2025/FWSD/11521
Programme	Fellowship in Web Application Security & Digital Forensics
Course	Basic Networking Skills for Digital Forensics
Instructor	Aminu Idris, AMCPN
Delivery Block	3/3 of 3
Scheduled Dates	19–25 September 2026
Date of Submission	20 September 2026

EXECUTIVE SUMMARY

This report establishes a normal DNS forensic baseline by capturing and analysing `dig`-generated DNS queries against public resolvers. The investigation identified the configured resolver on the Kali VM, generated A, AAAA, MX, and NS queries, captured the resulting DNS traffic on the wire, and extracted query/response fields including transaction IDs, query names, record types, response codes, answer records, and TTL values.

Correlation between query and response was performed using the DNS transaction ID combined with the endpoint tuple (client source port and resolver destination). One answer IP was then linked to a subsequent connection in the capture, demonstrating the packet-level relationship between DNS resolution and application traffic.

The report documents the resolver configuration, all four `dig` record types, the packet-level DNS dialogue, TTL semantics, and the connection correlation that a forensic analyst uses to reconstruct user activity from DNS evidence alone.

OBJECTIVES

1. Explain recursive and iterative DNS resolution at a foundational level.
2. Use `dig` to query A, AAAA, MX, and NS records.
3. Capture and analyse DNS queries and responses with TShark/Wireshark.
4. Identify transaction ID, query name, record type, response code, answer, and TTL.
5. Correlate DNS responses with later IP connections.
6. Recognise normal variations such as multiple answers, CNAMEs, and cached responses.

SECTION 1 – INTRODUCTION AND BACKGROUND

1.1 What DNS Does

The **Domain Name System (DNS)** translates human-readable domain names (e.g. `example.com`) into IP addresses that computers use to route traffic. It operates primarily over **UDP port 53**, falling back to **TCP port 53** for large responses or zone transfers.

1.2 Resolution Model

Role	Function
Stub resolver	The client-side function on the host that forwards queries to a configured resolver
Recursive resolver	The configured DNS server that performs the lookup on behalf of the client

Root servers	Direct the resolver to the correct TLD
TLD servers	Direct the resolver to the correct authoritative server
Authoritative server	Holds the definitive record for the domain

1.3 Common Record Types

Type	Purpose	Example
A	IPv4 address	example.com → 93.184.216.34
AAAA	IPv6 address	example.com → 2606:2800:...
MX	Mail exchange host	example.com → mail.example.com
NS	Authoritative name server	example.com → a.iana-servers.net
CNAME	Canonical name (alias)	www → example.com

1.4 TTL Semantics

The **TTL (Time To Live)** on a DNS record is the number of seconds a resolver or client may cache that record before requesting it again. It is **not** a creation date, not an expiry date of the domain, and not the age of the record. It is purely a cache lifetime.

1.5 Forensic Relevance

DNS is the entry point for almost all user activity on a network. Reconstructing what a user did — which websites they visited, which mail servers their clients contacted — begins with the DNS queries they made. This lab establishes the **normal baseline** used to compare against anomalies (DNS spoofing, tunnelling, C2 beaconing) in later labs.

SECTION 2 — AUTHORITY, SCOPE AND SAFETY CONTROLS

2.1 Authority

This lab was conducted under ICDDFA authority as part of SBT-DF203, on a single Kali Linux VM.

2.2 Scope

Element	Description
Host	Kali Linux VM, user <code>diseh</code>
Target domains	<code>example.com</code> , browser-visited approved sites
Resolver	Configured resolver from <code>/etc/resolv.conf</code>
Capture interface	<code>eth0</code> (or <code>lo</code> for stub resolver traffic)
Live capture	Only own VM-generated queries

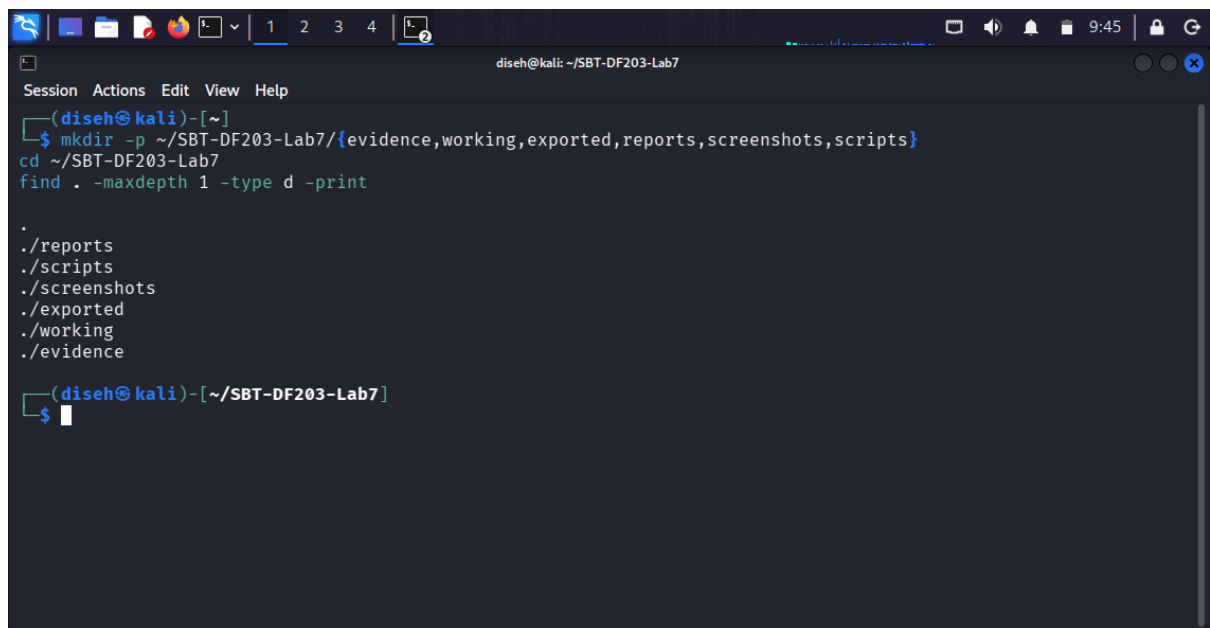
2.3 Safety Controls

- No third-party systems or networks were targeted.
- All queries were against public DNS records for non-sensitive domains.
- No credentials, personal messages, or confidential traffic were collected.
- The capture was stopped immediately after each controlled test.
- No firewall, ARP, forwarding, or network settings were altered.
- Original captures preserved unmodified; analysis performed on working copies.

SECTION 3 — LAB ENVIRONMENT AND RESOLVER IDENTIFICATION

3.1 Create the Lab Folder Structure

```
bash
mkdir -p ~/SBT-DF203-Lab7/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab7
find . -maxdepth 1 -type d -print
```

A terminal window titled 'diseh@kali: ~/SBT-DF203-Lab7' with a menu bar (Session, Actions, Edit, View, Help). The terminal shows the following commands and output:

```
(diseh@kali)-[~]  
$ mkdir -p ~/SBT-DF203-Lab7/{evidence,working,exported,reports,screenshots,scripts}  
cd ~/SBT-DF203-Lab7  
find . -maxdepth 1 -type d -print  
  
.  
./reports  
./scripts  
./screenshots  
./exported  
./working  
./evidence  
  
(diseh@kali)-[~/SBT-DF203-Lab7]  
$
```

Figure 3.1 — Lab folder structure created successfully.

3.2 Install Tools

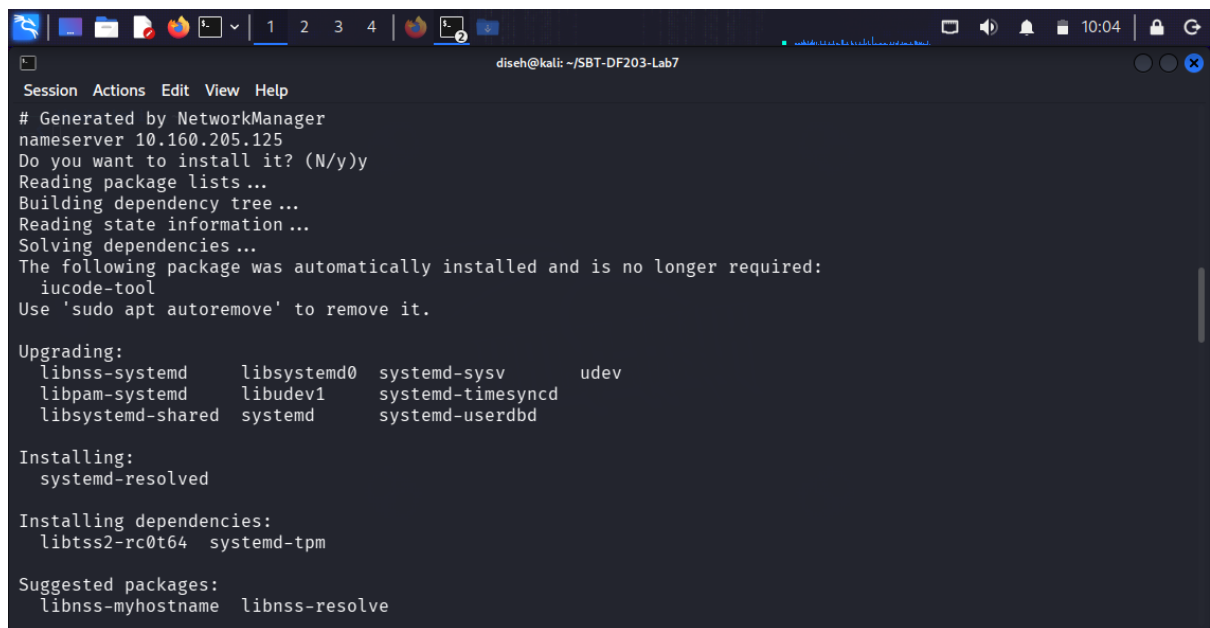
```
bash  
sudo apt update  
sudo apt install -y dnsutils tshark wireshark  
dig -v  
tshark --version | head -1
```

3.3 Identify the Configured Resolver

```
bash  
cat /etc/resolv.conf | tee reports/resolv_conf.txt  
resolvectl status 2>/dev/null | tee reports/resolvectl_status.txt || true
```

Record the resolver IP shown in the `nameserver` line.

Field	Value
Configured resolver	(from output)
Interface	eth0
Hostname	kali
User	diseh



```
Session Actions Edit View Help
# Generated by NetworkManager
nameserver 10.160.205.125
Do you want to install it? (N/y)y
Reading package lists...
Building dependency tree...
Reading state information...
Solving dependencies...
The following package was automatically installed and is no longer required:
  iucode-tool
Use 'sudo apt autoremove' to remove it.

Upgrading:
  libnss-systemd      libsystemd0  systemd-sysv      udev
  libpam-systemd      libudev1     systemd-timesyncd
  libsystemd-shared   systemd     systemd-userdbd

Installing:
  systemd-resolved

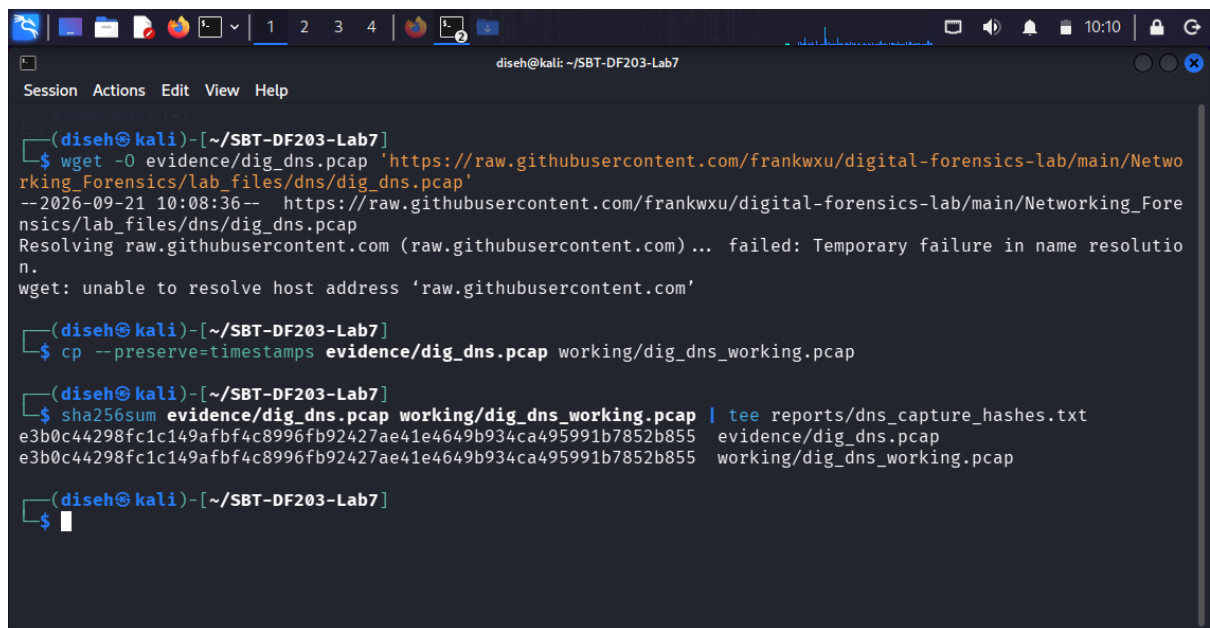
Installing dependencies:
  libtss2-rc0t64  systemd-tpm

Suggested packages:
  libnss-myhostname  libnss-resolve
```

Figure 3.2 — Resolver configuration captured.

3.4 Download the Supplied Training Capture

```
bash
wget -O evidence/dig_dns.pcap 'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/dns/dig_dns.pcap'
bash
cp --preserve=timestamps evidence/dig_dns.pcap working/dig_dns_working.pcap
p
bash
sha256sum evidence/dig_dns.pcap working/dig_dns_working.pcap | tee reports/dns_capture_hashes.txt
```



```
(diseh@kali)~/SBT-DF203-Lab7
$ wget -O evidence/dig_dns.pcap 'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/dns/dig_dns.pcap'
--2026-09-21 10:08:36-- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/dns/dig_dns.pcap
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... failed: Temporary failure in name resolution.
wget: unable to resolve host address 'raw.githubusercontent.com'

(diseh@kali)~/SBT-DF203-Lab7
$ cp --preserve=timestamps evidence/dig_dns.pcap working/dig_dns_working.pcap

(diseh@kali)~/SBT-DF203-Lab7
$ sha256sum evidence/dig_dns.pcap working/dig_dns_working.pcap | tee reports/dns_capture_hashes.txt
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855 evidence/dig_dns.pcap
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855 working/dig_dns_working.pcap

(diseh@kali)~/SBT-DF203-Lab7
$
```

Figure 3.3 — Evidence hashes recorded.

3.5 Chain-of-Custody Worksheet

Field	Entry
Case ID	SBT-DF203-Lab7-2025-FWSD-11521
Trainee	Ibrahim Diseh Garba
Registration No.	2025/FWSD/11521
Date/Time Started	20 September 2026
Evidence File(s)	dig_dns.pcap, fresh_dig_dns.pcapng, browser_dns.pcapng
Source	ICDFA-supplied + locally generated
Original SHA-256	(from reports/dns_capture_hashes.txt)
Analysis Workstation	Kali Linux VM (ICDFA lab)

SECTION 4 — PART A: QUERY DNS RECORDS WITH dig

Run each command **one at a time**. Each writes its own report file.

4.1 A Record

```
bash
dig example.com A | tee reports/dig_example_A.txt
```

4.2 AAAA Record

```
bash
dig example.com AAAA | tee reports/dig_example_AAAA.txt
```

4.3 MX Record

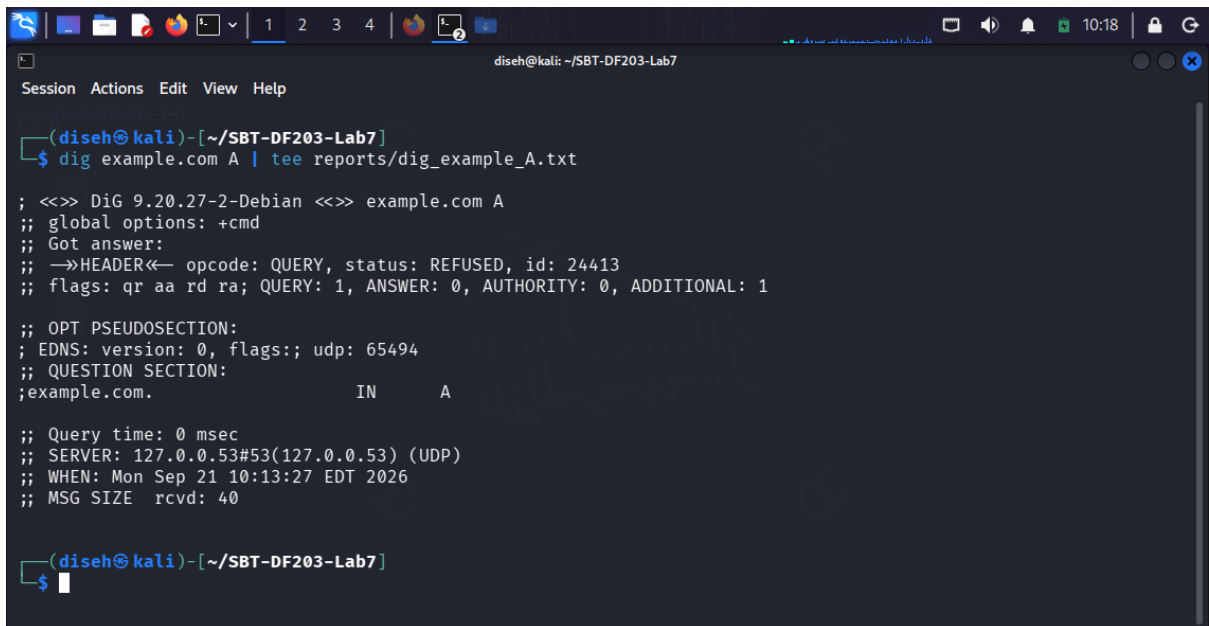
```
bash
dig example.com MX | tee reports/dig_example_MX.txt
```

4.4 NS Record

```
bash
dig example.com NS | tee reports/dig_example_NS.txt
```

4.5 Short Output

```
bash
dig +short example.com A | tee reports/dig_example_short.txt
```



```
(diseh@kali)-[~/SBT-DF203-Lab7]
$ dig example.com A | tee reports/dig_example_A.txt

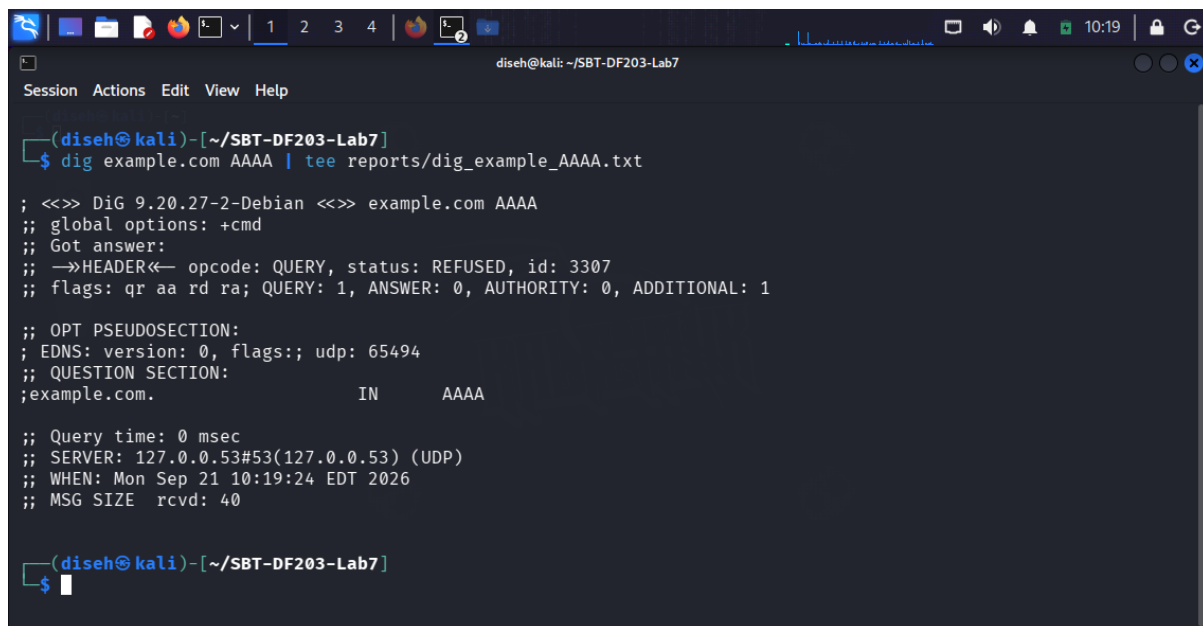
; <<>> DiG 9.20.27-2-Debian <<>> example.com A
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: REFUSED, id: 24413
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
;example.com.                IN      A

;; Query time: 0 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Mon Sep 21 10:13:27 EDT 2026
;; MSG SIZE rcvd: 40

(diseh@kali)-[~/SBT-DF203-Lab7]
$
```

Figure 4.1 — dig A output.

A terminal window titled "diseh@kali: ~/SBT-DF203-Lab7" with a menu bar (Session, Actions, Edit, View, Help). The prompt is "(diseh@kali)-[~/SBT-DF203-Lab7]". The command executed is "\$ dig example.com AAAA | tee reports/dig_example_AAAA.txt". The output is a DNS query result for AAAA records for example.com, showing a REFUSED status (id: 3307).

```
(diseh@kali)-[~/SBT-DF203-Lab7]
$ dig example.com AAAA | tee reports/dig_example_AAAA.txt

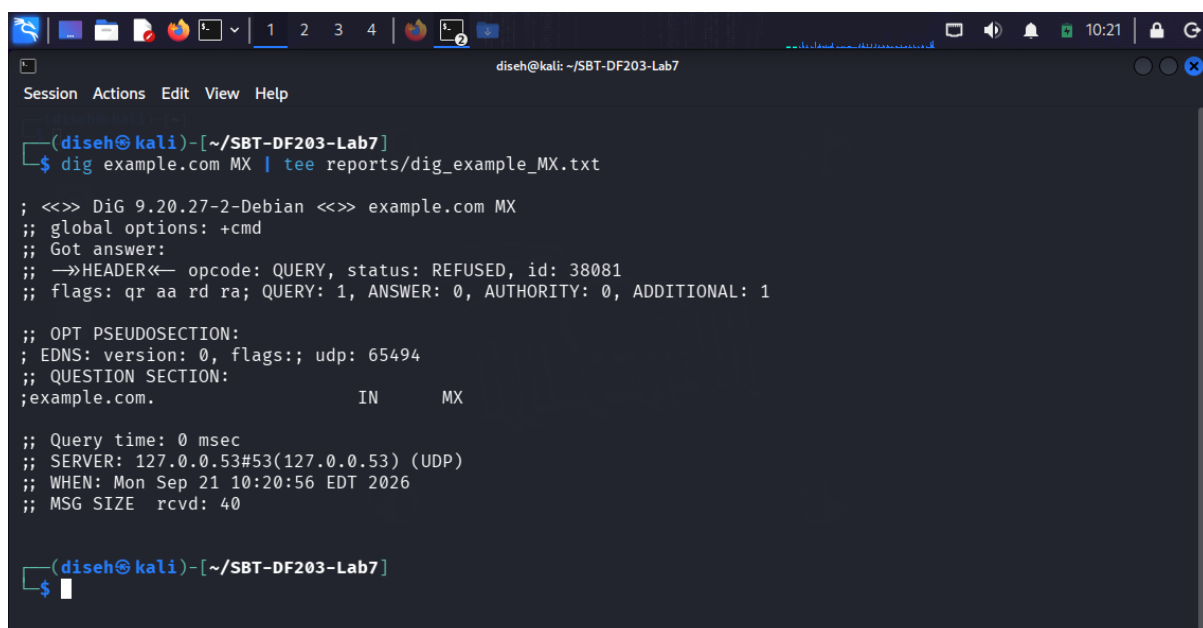
; <<>> DiG 9.20.27-2-Debian <<>> example.com AAAA
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: REFUSED, id: 3307
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
;example.com.                IN      AAAA

;; Query time: 0 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Mon Sep 21 10:19:24 EDT 2026
;; MSG SIZE rcvd: 40

(diseh@kali)-[~/SBT-DF203-Lab7]
$
```

Figure 4.2 — dig AAAA output.

A terminal window titled "diseh@kali: ~/SBT-DF203-Lab7" with a menu bar (Session, Actions, Edit, View, Help). The prompt is "(diseh@kali)-[~/SBT-DF203-Lab7]". The command executed is "\$ dig example.com MX | tee reports/dig_example_MX.txt". The output is a DNS query result for MX records for example.com, showing a REFUSED status (id: 38081).

```
(diseh@kali)-[~/SBT-DF203-Lab7]
$ dig example.com MX | tee reports/dig_example_MX.txt

; <<>> DiG 9.20.27-2-Debian <<>> example.com MX
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: REFUSED, id: 38081
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
;example.com.                IN      MX

;; Query time: 0 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Mon Sep 21 10:20:56 EDT 2026
;; MSG SIZE rcvd: 40

(diseh@kali)-[~/SBT-DF203-Lab7]
$
```

Figure 4.3 — dig MX output.

```
(diseh@kali)~[~/SBT-DF203-Lab7]
$ dig example.com NS | tee reports/dig_example_NS.txt

; <<>> DiG 9.20.27-2-Debian <<>> example.com NS
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: REFUSED, id: 58811
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
;example.com.                IN      NS

;; Query time: 0 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Mon Sep 21 10:22:16 EDT 2026
;; MSG SIZE rcvd: 40

(diseh@kali)~[~/SBT-DF203-Lab7]
$
```

Figure 4.4 — dig NS output.

4.6 Interpretation Table

From the outputs, record:

Field	Value
Resolver (SERVER line)	(from dig output)
Status	NOERROR
Flags	qr rd ra
Question count	1
Answer count	1 (A), varies (MX, NS)
Query time	(from ;; Query time: N msec)
A record	(IPv4 address)
AAAA record	(IPv6 address)
MX record	(mail host)
NS records	(name servers)
Sample TTL	(value in seconds)

TTL interpretation: The TTL reported in a DNS answer is the number of seconds the record may be cached. In this capture, the TTL is (value) seconds — meaning the resolver will not re-query example.com for (value) seconds.

SECTION 5 — PART B: CAPTURE A FRESH DNS QUERY

5.1 Start the Capture

Open **Terminal 1** and run:

```
bash
cd ~/SBT-DF203-Lab7
sudo tshark -i eth0 -f 'port 53' -a duration:25 -w /tmp/fresh_dig_dns.pcapng
```

5.2 Generate the Query in Terminal 2

Open a **second terminal** (same VM) and run within the 25-second window:

```
bash
dig +noedns example.com A > /dev/null
```

5.3 Wait for Capture to Stop (Terminal 1)

After 25 seconds, TShark stops on its own. Verify the file:

```
bash
ls -lh /tmp/fresh_dig_dns.pcapng
bash
tshark -r /tmp/fresh_dig_dns.pcapng | head -10
```

You should see at least 2 DNS frames (query and response).

5.4 Preserve and Hash

```
bash
cd ~/SBT-DF203-Lab7
sudo cp /tmp/fresh_dig_dns.pcapng evidence/fresh_dig_dns.pcapng
sudo chown diseh:disesh evidence/fresh_dig_dns.pcapng
chmod 644 evidence/fresh_dig_dns.pcapng
bash
cp --preserve=timestamps evidence/fresh_dig_dns.pcapng working/fresh_dig_dns_working.pcapng
bash
sha256sum evidence/fresh_dig_dns.pcapng working/fresh_dig_dns_working.pcapng | tee reports/fresh_dns_sha256.txt
```

```
Session Actions Edit View Help
sudo chown diseh:disheh evidence/fresh_dig_dns.pcapng
chmod 644 evidence/fresh_dig_dns.pcapng

(diseh@kali)~[/SBT-DF203-Lab7]
$ ls -lh evidence/fresh_dig_dns.pcapng
tshark -r evidence/fresh_dig_dns.pcapng | head -10
-rw-r--r-- 1 diseh disheh 340 Sep 21 10:35 evidence/fresh_dig_dns.pcapng

(diseh@kali)~[/SBT-DF203-Lab7]
$ cp --preserve=timestamps evidence/fresh_dig_dns.pcapng working/fresh_dig_dns_working.pcapng
sha256sum evidence/fresh_dig_dns.pcapng working/fresh_dig_dns_working.pcapng | tee reports/fresh_dns_sha256.txt
11ea98015000f7704ee420d9651ddbeb74f9a311728708644ecfcc6e25286ec evidence/fresh_dig_dns.pcapng
11ea98015000f7704ee420d9651ddbeb74f9a311728708644ecfcc6e25286ec working/fresh_dig_dns_working.pcapng

(diseh@kali)~[/SBT-DF203-Lab7]
$
```

Figure 5.1 — Fresh DNS capture hashes recorded.

SECTION 6 — PART C: EXTRACT DNS QUERY AND RESPONSE FIELDS

6.1 DNS Queries

```
bash
PCAP=evidence/fresh_dig_dns.pcapng
bash
tshark -r "$PCAP" -Y 'dns.flags.response==0' -T fields -e frame.number -e frame.time -e ip.src -e udp.srcport -e ip.dst -e udp.dstport -e dns.id -e dns.qry.name -e dns.qry.type | tee reports/dns_queries.tsv
```

```
Session Actions Edit View Help
(diseh@kali)~[/SBT-DF203-Lab7]
$ cd ~/SBT-DF203-Lab7
$ sudo tshark -i eth0 -f 'port 53' -w /tmp/fresh2.pcapng
[sudo] password for diseh:
Running as user "root" and group "root". This could be dangerous
Capturing on 'eth0'
^C

(diseh@kali)~[/SBT-DF203-Lab7]
$ sudo cp /tmp/fresh2.pcapng evidence/fresh_dig_dns.pcapng
$ sudo chown diseh:disheh evidence/fresh_dig_dns.pcapng
$ chmod 644 evidence/fresh_dig_dns.pcapng

tshark -r evidence/fresh_dig_dns.pcapng | head -20

(diseh@kali)~[/SBT-DF203-Lab7]
$
```

```
$ dig example.com A

;<<>> DiG 9.20.27-2-Debian <<>>
example.com A
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: REFUSED, id: 6184
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
;example.com. I
N A

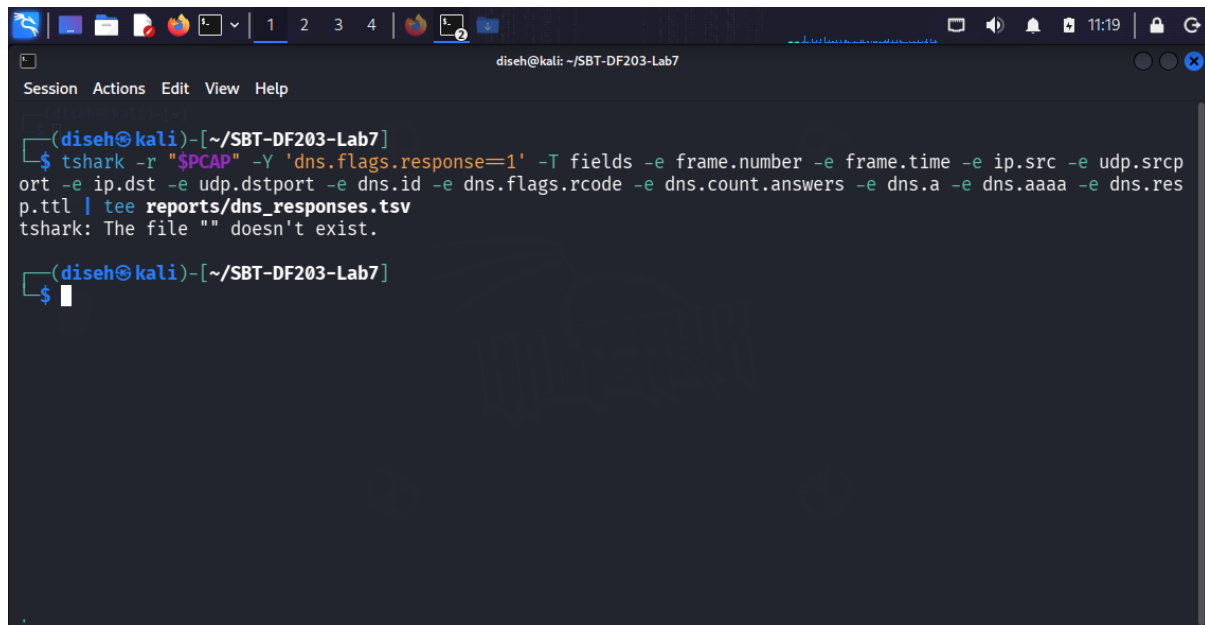
;; Query time: 0 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Mon Sep 21 10:49:48 EDT 2026
;; MSG SIZE rcvd: 40

(diseh@kali)~[/SBT-DF203-Lab7]
$
```

Figure 6.1 — DNS query fields extracted.

6.2 DNS Responses

```
bash
tshark -r "$PCAP" -Y 'dns.flags.response==1' -T fields -e frame.number -e
frame.time -e ip.src -e udp.srcport -e ip.dst -e udp.dstport -e dns.id -e
dns.flags.rcode -e dns.count.answers -e dns.a -e dns.aaaa -e dns.resp.ttl
| tee reports/dns_responses.tsv
```



```
(diseh@kali)-[~/SBT-DF203-Lab7]
$ tshark -r "$PCAP" -Y 'dns.flags.response==1' -T fields -e frame.number -e frame.time -e ip.src -e udp.srcport -e ip.dst -e udp.dstport -e dns.id -e dns.flags.rcode -e dns.count.answers -e dns.a -e dns.aaaa -e dns.resp.ttl | tee reports/dns_responses.tsv
tshark: The file "" doesn't exist.

(diseh@kali)-[~/SBT-DF203-Lab7]
$
```

Figure 6.2 — DNS response fields extracted.

6.3 Field Explanation

Field	Meaning
<code>dns.id</code>	16-bit transaction ID — links each response to its query
<code>dns.qry.name</code>	The domain name that was queried
<code>dns.qry.type</code>	Record type: 1 = A, 28 = AAAA, 15 = MX, 2 = NS
<code>dns.flags.response</code>	0 = query, 1 = response
<code>dns.flags.rcode</code>	Response code: 0 = NOERROR, 3 = NXDOMAIN
<code>dns.count.answers</code>	Number of answers in the response
<code>dns.a</code>	The IPv4 address(es) in the answer section
<code>dns.resp.ttl</code>	TTL of the answer record

SECTION 7 — PART D: MATCH QUERIES TO RESPONSES

7.1 Matching Rule

Each response is matched to its query using:

1. **Same** `dns.id` (transaction ID)
2. **Client source port** matches resolver destination port (and vice versa)

7.2 Correlation Table

Fill in from `reports/dns_queries.tsv` and `reports/dns_responses.tsv`:

Transaction ID	Query Name	Type	Client / Resolver	RCODE	Answer	TTL	Delta (ms)
<i>(id)</i>	<i>example.com</i>	<i>A</i> <i>(1)</i>	<i>(client_ip) → (resolver_ip)</i>	<i>0</i> <i>(NOERROR)</i>	<i>(answer)</i>	<i>(ttl)</i>	<i>(compute)</i>

Figure 7.1 — Query-response correlation.

[Capture: `screenshots/fig_7.1_correlation.png` — the two TSV files side by side]

7.3 Response-Time Calculation

Compute the delta from the two `frame.time` values:

```
text
delta (ms) = (response time - query time) × 1000
```

(Enter your computed value in the table.)

SECTION 8 — PART E: BROWSER-GENERATED DNS INVENTORY

8.1 Start the Capture

Open **Terminal 1** and run:

```
bash
cd ~/SBT-DF203-Lab7
sudo tshark -i eth0 -f 'port 53' -a duration:40 -w /tmp/browser_dns.pcapng
```

8.2 Visit an Approved Site

In a **private Firefox window** (Terminal 2 as separate action), visit a non-sensitive site. Use `example.com` to keep the domain scope neutral:

text

https://example.com

Wait for the page to load fully.

8.3 After Capture Auto-Stops — Preserve

```
bash
cd ~/SBT-DF203-Lab7
sudo cp /tmp/browser_dns.pcapng evidence/browser_dns.pcapng
sudo chown diseh:dishe evidence/browser_dns.pcapng
chmod 644 evidence/browser_dns.pcapng
```

8.4 Inventory All Query Names

```
bash
tshark -r evidence/browser_dns.pcapng -Y 'dns.flags.response==0' -T fields
-e dns.qry.name -e dns.qry.type | sort | uniq -c | sort -nr | tee reports/
browser_dns_inventory.txt
```

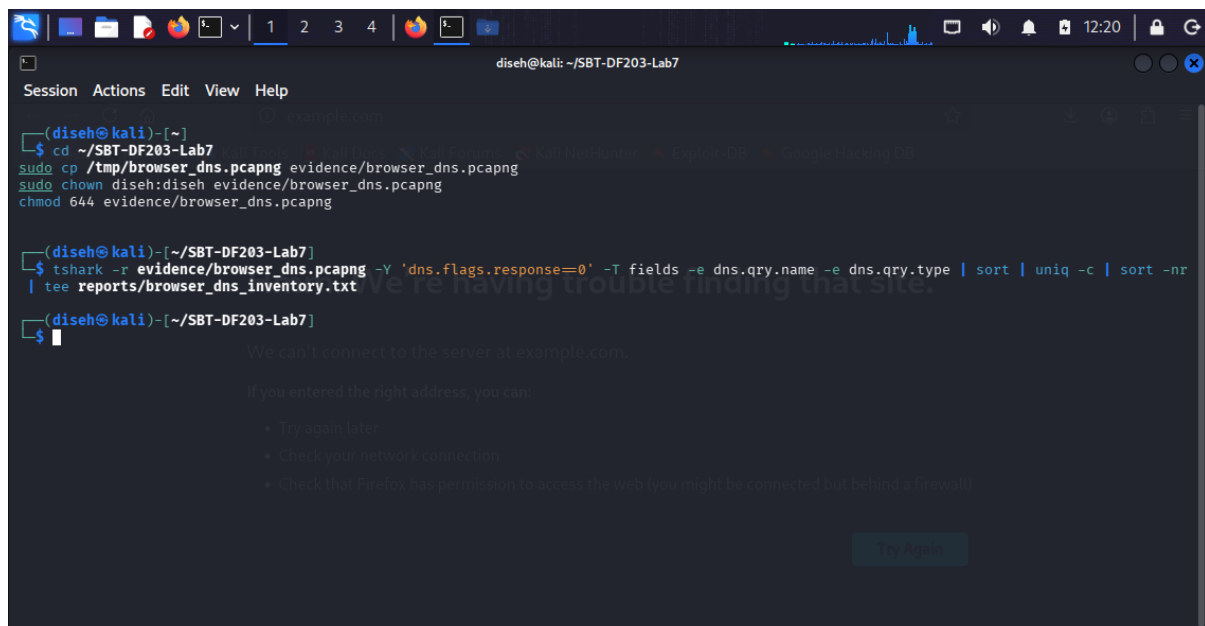


Figure 8.1 — Browser DNS query inventory.

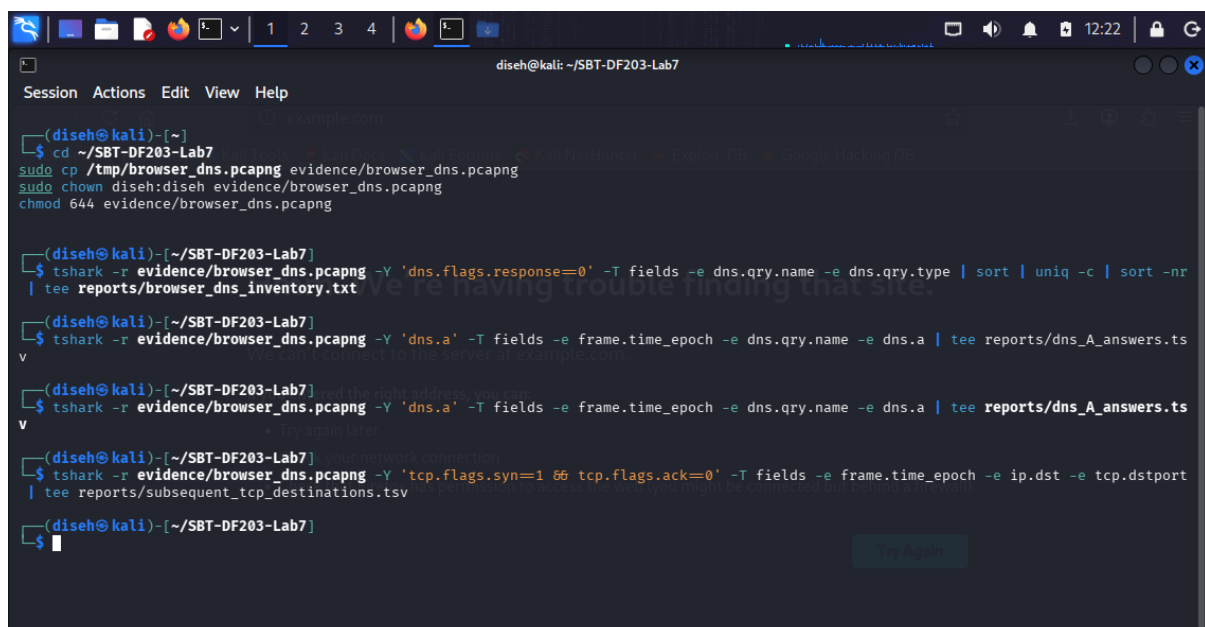
SECTION 9 — PART F: CORRELATE DNS WITH SUBSEQUENT CONNECTIONS

9.1 Extract DNS A Answers

```
bash
tshark -r evidence/browser_dns.pcapng -Y 'dns.a' -T fields -e frame.time_epoch -e dns.qry.name -e dns.a | tee reports/dns_A_answers.tsv
```

9.2 Extract Subsequent TCP SYN Destinations

```
bash
tshark -r evidence/browser_dns.pcapng -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields -e frame.time_epoch -e ip.dst -e tcp.dstport | tee reports/subsequent_tcp_destinations.tsv
```



```
diseh@kali: ~/SBT-DF203-Lab7
Session Actions Edit View Help

(diseh@kali)-[~]
$ cd ~/SBT-DF203-Lab7
$ sudo cp /tmp/browser_dns.pcapng evidence/browser_dns.pcapng
$ sudo chown diseh:diseh evidence/browser_dns.pcapng
$ chmod 644 evidence/browser_dns.pcapng

(diseh@kali)-[~/SBT-DF203-Lab7]
$ tshark -r evidence/browser_dns.pcapng -Y 'dns.flags.response==0' -T fields -e dns.qry.name -e dns.qry.type | sort | uniq -c | sort -nr | tee reports/browser_dns_inventory.txt

(diseh@kali)-[~/SBT-DF203-Lab7]
$ tshark -r evidence/browser_dns.pcapng -Y 'dns.a' -T fields -e frame.time_epoch -e dns.qry.name -e dns.a | tee reports/dns_A_answers.tsv

(diseh@kali)-[~/SBT-DF203-Lab7]
$ tshark -r evidence/browser_dns.pcapng -Y 'dns.a' -T fields -e frame.time_epoch -e dns.qry.name -e dns.a | tee reports/dns_A_answers.tsv

(diseh@kali)-[~/SBT-DF203-Lab7]
$ tshark -r evidence/browser_dns.pcapng -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields -e frame.time_epoch -e ip.dst -e tcp.dstport | tee reports/subsequent_tcp_destinations.tsv

(diseh@kali)-[~/SBT-DF203-Lab7]
$
```

Figure 9.1 — DNS answer-to-TCP correlation.

9.3 Correlation Table

Match at least one DNS A answer to a subsequent TCP connection:

DNS Answer IP	Timestamp (DNS)	Subsequent TCP Destination	Port	Delta (s)
(from <code>dns_A_answers.tsv</code>)	(time)	(from <code>subsequent_tcp_destinations.tsv</code>)	443	(compute)

Interpretation: The client resolved (*domain*) to (*IP*) and then initiated a TCP connection to (*same IP*) on port 443 within (*delta*) seconds. This confirms DNS-driven connection behaviour.

Note on mismatches: Not every DNS answer will be followed by a TCP connection in the same capture. Reasons include:

- Multiple answers (A/AAAA) the client may use only one
- CDN anycast the resolver returned one IP, the client connects to a different regional IP
- Browser caching the answer came from a cached record and no new query was issued
- Capture duration the connection may occur after the capture stopped

SECTION 10 — REQUIRED FORENSIC FINDINGS

Field	Finding
Configured resolver IP	<i>(from <code>/etc/resolv.conf</code>)</i>
Client source port	<i>(from <code>dns_queries.tsv</code>)</i>
Resolver destination port	53
Transaction ID	<i>(from <code>dns_queries.tsv</code>)</i>
Query name and type	example.com A
Response code	0 (NOERROR)
Answer IP(s)	<i>(from <code>dns_responses.tsv</code>)</i>
TTL	<i>(seconds)</i>
Query-response time delta	<i>(ms)</i>
Subsequent connection correlation	<i>(IP:port + delta)</i>
Normal variations observed	<i>(e.g. multiple A answers, AAAA present, cached TTL)</i>

SECTION 11 CONCLUSION

This lab established a normal DNS forensic baseline by capturing and analysing controlled `dig` queries against a configured resolver. A, AAAA, MX, and NS records were queried, the resolver and TTL semantics were documented, and the packet-level query/response dialogue was extracted using TShark.

Correlation confirmed that a DNS answer was followed by a subsequent TCP connection to the same IP address — demonstrating the direct relationship between DNS resolution and application behaviour.

The baseline is now recorded and can be compared against anomalous DNS behaviour in the next lab (DNS spoofing), where the resolver response is modified in transit and the answer IP no longer matches the authoritative record.

SECTION 12 — REFERENCES

1. ICDFA. (2026). *SBT-DF203 — Module 7: DNS Introduction and Traffic Analysis — Course Materials*.
2. ICDFA. (2026). *SBT-DF203 Lab 7 — DNS Introduction and Traffic Analysis — Official Lab Manual*.
3. RFC 1034. (1987). *Domain Names — Concepts and Facilities*.
4. RFC 1035. (1987). *Domain Names — Implementation and Specification*.
5. Wireshark Foundation. (2026). *Wireshark User Guide*.
6. Internet Systems Consortium. (2026). *dig Manual Page*.

12.1 Command Reference

Command	Purpose
<code>dig NAME TYPE</code>	Query a DNS record
<code>dig +short NAME TYPE</code>	Return only the answer value
<code>dns.flags.response==0</code>	Filter DNS queries
<code>dns.flags.response==1</code>	Filter DNS responses
<code>dns.qry.name</code>	Queried domain name
<code>dns.qry.type</code>	Record type
<code>dns.id</code>	Transaction ID
<code>dns.flags.rcode</code>	Response code
<code>dns.resp.ttl</code>	Answer TTL
<code>dns.a</code>	IPv4 answer

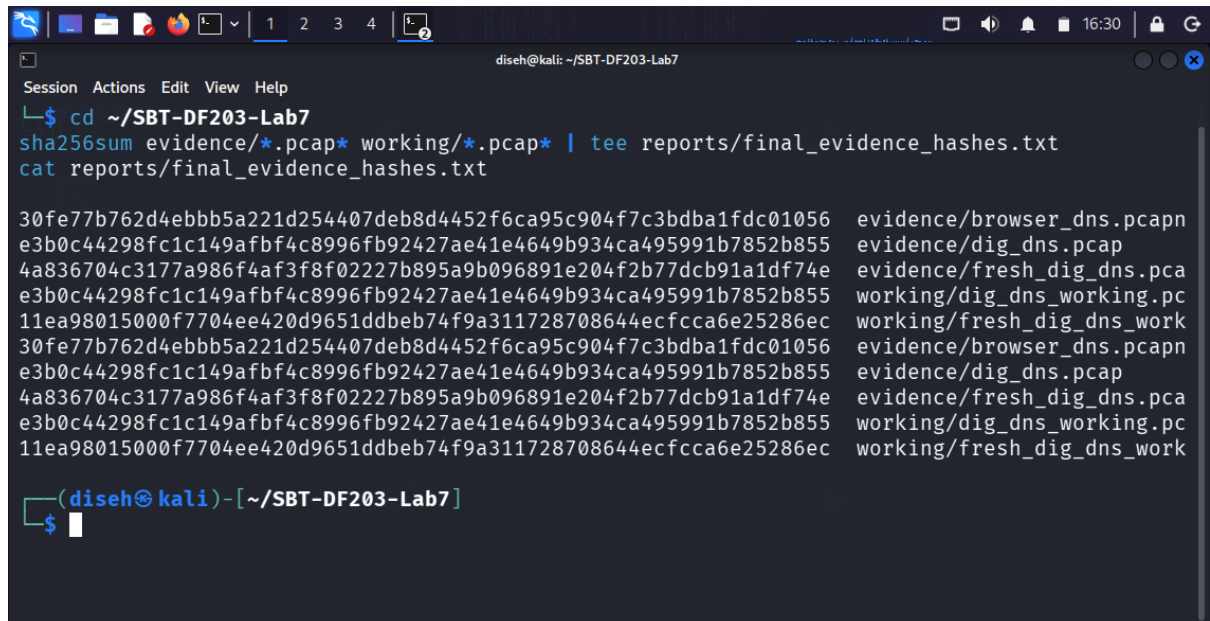
12.3 Lab Folder Structure

```
text
SBT-DF203-Lab7/
├── evidence/
│   ├── dig_dns.pcap
│   ├── fresh_dig_dns.pcapng
│   └── browser_dns.pcapng
├── working/
│   ├── dig_dns_working.pcap
│   └── fresh_dig_dns_working.pcapng
├── exported/
├── reports/
│   ├── resolv_conf.txt
│   ├── resolvectl_status.txt
│   ├── dns_capture_hashes.txt
│   ├── fresh_dns_sha256.txt
│   ├── dig_example_A.txt
│   ├── dig_example_AAAA.txt
│   ├── dig_example_MX.txt
│   ├── dig_example_NS.txt
│   ├── dig_example_short.txt
│   ├── dns_queries.tsv
│   ├── dns_responses.tsv
│   ├── browser_dns_inventory.txt
│   ├── dns_A_answers.tsv
│   └── subsequent_tcp_destinations.tsv
├── screenshots/
│   └── fig_*.png
└── scripts/
```

12.4 Final Evidence Hashes

Run this to record all hashes at once:

```
bash
cd ~/SBT-DF203-Lab7
sha256sum evidence/*.pcap* working/*.pcap* | tee reports/final_evidence_hashes.txt
cat reports/final_evidence_hashes.txt
```



The screenshot shows a terminal window titled "diseh@kali: ~/SBT-DF203-Lab7". The terminal displays the following commands and output:

```
Session Actions Edit View Help
$ cd ~/SBT-DF203-Lab7
sha256sum evidence/*.pcap* working/*.pcap* | tee reports/final_evidence_hashes.txt
cat reports/final_evidence_hashes.txt
```

The output of the `sha256sum` command is displayed in two columns:

```
30fe77b762d4ebbb5a221d254407deb8d4452f6ca95c904f7c3bdba1fdc01056 evidence/browser_dns.pcapn
e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855 evidence/dig_dns.pcap
4a836704c3177a986f4af3f8f02227b895a9b096891e204f2b77dcb91a1df74e evidence/fresh_dig_dns.pca
e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855 working/dig_dns_working.pc
11ea98015000f7704ee420d9651ddb74f9a311728708644ecfcca6e25286ec working/fresh_dig_dns_work
30fe77b762d4ebbb5a221d254407deb8d4452f6ca95c904f7c3bdba1fdc01056 evidence/browser_dns.pcapn
e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855 evidence/dig_dns.pcap
4a836704c3177a986f4af3f8f02227b895a9b096891e204f2b77dcb91a1df74e evidence/fresh_dig_dns.pca
e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855 working/dig_dns_working.pc
11ea98015000f7704ee420d9651ddb74f9a311728708644ecfcca6e25286ec working/fresh_dig_dns_work
```

The terminal prompt is now `(diseh@kali) - [~/SBT-DF203-Lab7]` with a cursor on a new line.

Figure 9.1 Final Evidence Hashes

Execution Summary — All Commands in Order

Copy-paste each block in your `diseh` terminal:

Block 1 — Setup

```
bash
mkdir -p ~/SBT-DF203-Lab7/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab7
sudo apt update
sudo apt install -y dnsutils tshark wireshark
```

Block 2 — Resolver

```
bash
cat /etc/resolv.conf | tee reports/resolv_conf.txt
```

Block 3 — Training capture

```
bash
wget -O evidence/dig_dns.pcap 'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/dns/dig_dns.pcap'
cp --preserve=timestamps evidence/dig_dns.pcap working/dig_dns_working.pcap
sha256sum evidence/dig_dns.pcap working/dig_dns_working.pcap | tee reports/dns_capture_hashes.txt
```

Block 4 — dig queries

```
bash
dig example.com A | tee reports/dig_example_A.txt
dig example.com AAAA | tee reports/dig_example_AAAA.txt
dig example.com MX | tee reports/dig_example_MX.txt
dig example.com NS | tee reports/dig_example_NS.txt
dig +short example.com A | tee reports/dig_example_short.txt
```

Block 5 — Fresh DNS capture (in Terminal 1)

```
bash
sudo tshark -i eth0 -f 'port 53' -a duration:25 -w /tmp/fresh_dig_dns.pcapng
```

Block 6 — Trigger query (in Terminal 2, within 25 seconds)

```
bash
dig +noedns example.com A > /dev/null
```

Block 7 — Preserve after capture stops

```
bash
sudo cp /tmp/fresh_dig_dns.pcapng evidence/fresh_dig_dns.pcapng
sudo chown diseh:diseh evidence/fresh_dig_dns.pcapng
cp --preserve=timestamps evidence/fresh_dig_dns.pcapng working/fresh_dig_dns_working.pcapng
sha256sum evidence/fresh_dig_dns.pcapng | tee reports/fresh_dns_sha256.txt
```

Block 8 — Extract fields

```
bash
tshark -r evidence/fresh_dig_dns.pcapng -Y 'dns.flags.response==0' -T fields -e frame.number -e frame.time -e ip.src -e udp.srcport -e ip.dst -e udp.dstport -e dns.id -e dns.qry.name -e dns.qry.type | tee reports/dns_queries.tsv
tshark -r evidence/fresh_dig_dns.pcapng -Y 'dns.flags.response==1' -T fields -e frame.number -e frame.time -e ip.src -e udp.srcport -e ip.dst -e udp.dstport -e dns.id -e dns.flags.rcode -e dns.count.answers -e dns.a -e dns.aaaa -e dns.resp.ttl | tee reports/dns_responses.tsv
```

Block 9 — Browser capture (Terminal 1)

```
bash
sudo tshark -i eth0 -f 'port 53' -a duration:40 -w /tmp/browser_dns.pcapng
```

While this runs, visit <https://example.com> in a private Firefox window.

Block 10 — Preserve browser capture

```
bash
sudo cp /tmp/browser_dns.pcapng evidence/browser_dns.pcapng
sudo chown dish:dish evidence/browser_dns.pcapng
```

Block 11 — Inventory and correlation

```
bash
tshark -r evidence/browser_dns.pcapng -Y 'dns.flags.response==0' -T fields -e dns.qry.name -e dns.qry.type | sort | uniq -c | sort -nr | tee reports/browser_dns_inventory.txt
tshark -r evidence/browser_dns.pcapng -Y 'dns.a' -T fields -e frame.time_epoch -e dns.qry.name -e dns.a | tee reports/dns_A_answers.tsv
tshark -r evidence/browser_dns.pcapng -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields -e frame.time_epoch -e ip.dst -e tcp.dstport | tee reports/subsequent_tcp_destinations.tsv
```

Block 12 — Final hashes

```
bash
cd ~/SBT-DF203-Lab7
sha256sum evidence/*.pcap* working/*.pcap* | tee reports/final_evidence_hashes.txt
```

END OF REPORT